```
        },
        "matched_queries" : [
          "filter_range_price_query",
          "should_match_energy_rating_query",
          "must_match_brand_query",
          "should_term_type_query"
        ]
      }
    ]
```

The results are matched on the four queries mentioned in the `matched_queries` block. The real benefit of naming queries is removing redundant queries not associated with the outcome. This way, we can reduce the query size and concentrate on tweaking the queries that are part of fetching the results.

This concludes our look at the `bool` query, which is one of the most important and sophisticated compound queries. In the next few sections, we look at other compound queries, starting with constant scores.

11.4 Constant scores

Previously, we looked at a `filter` query within a `bool` clause. For completeness, let's rerun a sample `filter` query to fetch products with a user rating between 4 and 5.

Listing 11.16 The `filter` clause declared in a `bool` query

```
GET products/_search
{
  "query": {
    "bool": {
      "filter": [
        {
          "range": {
            "user_ratings": {
              "gte": 4,
              "lte": 5
            }
          }
        }
      ]
    }
  }
}
```

The query results in all products matching the criteria for user ratings. The only point of interest is that the query is executed in a filter context; so, no score (zero) is associated with the results. However, we may need to set a nonzero score, especially when we want to boost a particular search criterion. This is where a new query type, `constant_score`, enters the picture.

As the name suggests, `constant_score` wraps a `filter` query and produces the results with a predefined (boosted) score. The query in the following listing shows this in action.